

Honeypots

Concepts, isolation et chaîne de collecte — le cours

Les trois familles de honeypots, par niveau d'interaction

	ce qu'il fait	risque	information obtenue
FAIBLE	simule quelques ports ouverts, rien de plus	aucun	peu d'information
MOYEN	simule un shell complet, n'exécute rien — Cowrie	faible	commandes, identifiants, fichiers tentés
FORT	vraie machine volontairement vulnérable	élevé : elle est réellement compromise	tout, y compris les 0-day

Cowrie est au milieu : assez riche pour être instructif, assez confiné pour être sûr.

Ce cours accompagne le TP 3. Il explique *pourquoi* les choix faits sur la machine sont les bons, et ce qu'il faut savoir avant d'exposer un honeypot sur Internet.

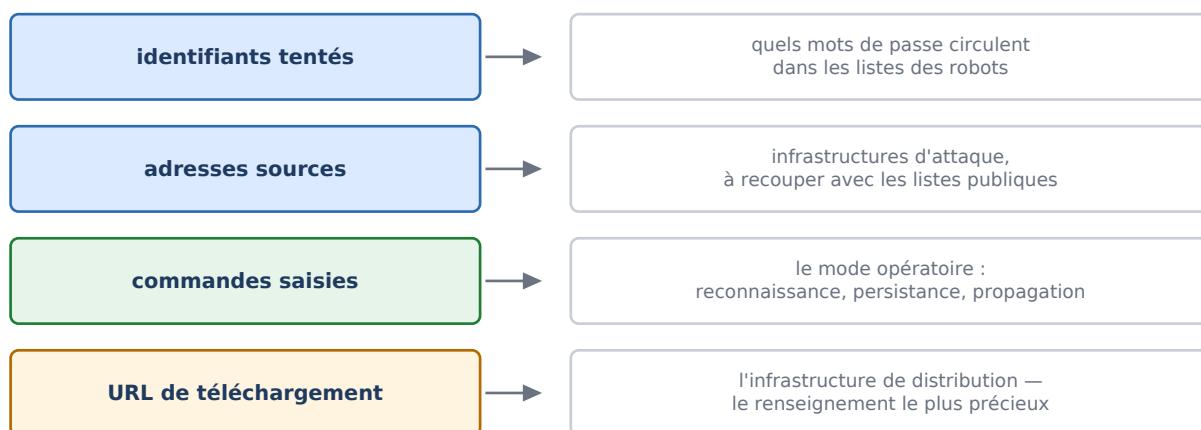
1. À quoi sert un honeypot

Un honeypot est un système **sans utilité légitime**, placé là pour être attaqué. C'est cette absence d'usage normal qui fait toute sa valeur :

Le principe fondateur : sur un serveur normal, distinguer une attaque du trafic légitime est difficile — c'est tout le problème de la détection d'intrusion, avec ses faux positifs. Sur un honeypot, il n'y a pas de trafic légitime. **Tout ce qui arrive est, par définition, suspect.** Le taux de faux positifs est nul.

Ce que ça apporte concrètement

Ce qu'on récolte, et pourquoi c'est utile



Les quatre catégories d'information qu'un honeypot SSH produit.

À quoi ça sert, dans l'ordre d'utilité croissante :

1. **Comprendre la pression réelle** sur une adresse IP exposée. La plupart des gens sous-estiment largement le volume : un port 22 ouvert sur Internet reçoit des milliers de tentatives par jour, dès les premières minutes.
2. **Connaître les identifiants attaqués**, donc savoir lesquels ne jamais utiliser.
3. **Observer les modes opératoires** : que fait l'attaquant *après* être entré ? La séquence est souvent identique (reconnaissance matérielle, désactivation de l'historique, téléchargement d'une charge, persistance) et très instructive.
4. **Récupérer les URL de distribution**, qui identifient les infrastructures utilisées.

Ce que ça n'apporte pas

- **Aucune protection.** Un honeypot ne défend rien. C'est un capteur, pas un bouclier.
- **Peu de ciblage.** On capture surtout des robots. Une attaque ciblée contre toi ne passera pas par là.
- **Un coût de surveillance.** Un honeypot dont personne ne lit les journaux est une machine exposée qui ne sert à rien — c'est-à-dire un risque net.

2. Les niveaux d'interaction

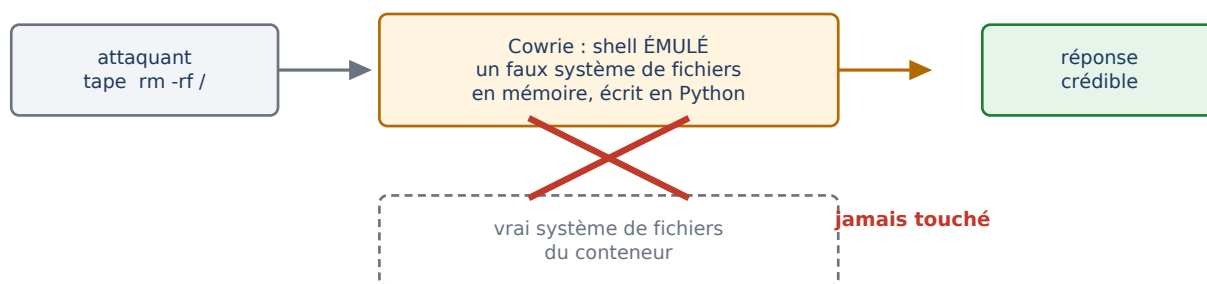
C'est la classification qui structure tout le domaine, et elle se résume à une question : **jusqu'où laisse-t-on l'attaquant aller ?**

Niveau	Principe	Exemples	Risque	Ce qu'on apprend
Faible	Ouvre des ports, répond des bannières. Aucune interaction réelle.	portspooft, honeyd	nul	Qui scanne, quels ports intéressent
Moyen	Émule un service complet — y compris un shell — mais n'exécute rien.	Cowrie , Dionaea	faible	Identifiants, commandes, charges tentées
Fort	Une vraie machine, réellement vulnérable et réellement compromise.	VM sacrificielle	élevé	Tout, y compris les vulnérabilités inconnues

Le choix de Cowrie se justifie par le rapport risque/information. Un honeypot à faible interaction ne dit presque rien : l'attaquant ne peut pas se connecter, donc on n'apprend que des scans. Un honeypot à forte interaction est une machine réellement compromise sur ton réseau, avec tout ce que cela implique — y compris ta responsabilité si elle sert à attaquer ailleurs. Cowrie donne l'essentiel de l'information avec une fraction du risque.

Comment Cowrie tient cette promesse

Pourquoi Cowrie est sûr : il n'exécute rien



L'attaquant voit un système entier. Il n'en existe aucun : tout est simulé.

Le shell est un programme Python qui imite un système. Aucune commande n'atteint le système de fichiers réel.

Quand l'attaquant tape `rm -rf /`, Cowrie ne transmet rien à un interpréteur : il consulte une arborescence factice chargée en mémoire, et renvoie une réponse plausible. Même chose pour `uname`, `cat /proc/cpuinfo`, `ps`. C'est un décor de théâtre, pas un système.

La limite à connaître : un décor se détecte. Un attaquant expérimenté teste des commandes inhabituelles, observe les incohérences de temps de réponse, vérifie que le contenu de `/proc` est cohérent. Cowrie arrête les robots et les opérateurs pressés ; il ne trompe pas quelqu'un qui cherche activement un honeypot. Ce n'est pas un défaut : c'est le compromis choisi.

3. La règle d'or : le confinement sortant

Un honeypot est fait pour être compromis. La seule question qui compte est : et après ? Toute la sécurité du dispositif tient dans ce qu'il peut atteindre une fois pris.

Un honeypot mal confiné devient, dans l'ordre de gravité :

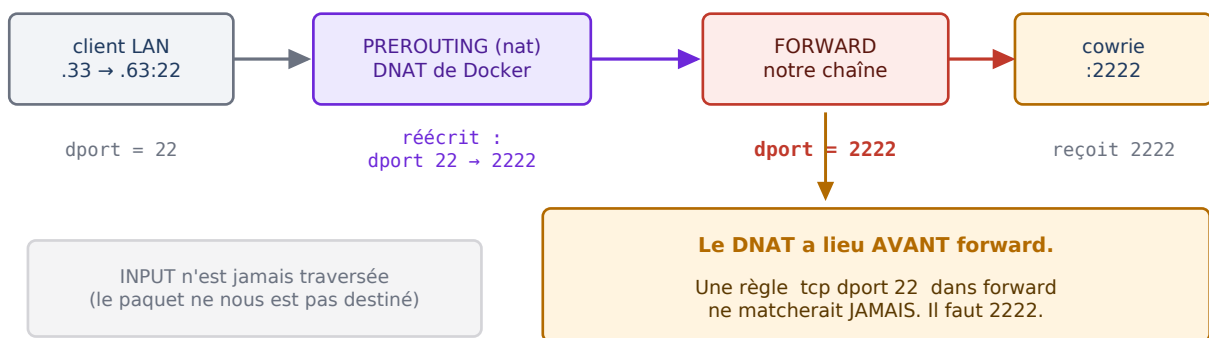
1. un **point d'appui** pour attaquer le reste de ton réseau — c'est le scénario le plus probable, car l'attaquant cherche systématiquement à rebondir ;
2. un **relais** pour attaquer des tiers : scan, déni de service, envoi de courrier indésirable. Le trafic part de ton adresse IP, avec ton nom d'abonné derrière ;
3. un **nœud de botnet** qui télécharge et exécute une charge.

D'où la règle, qui ne souffre aucune exception :

Sortant : tout est interdit, sauf ce qui est explicitement nécessaire. Dans notre cas, une seule destination : le collecteur de journaux, sur un seul port. Rien d'autre — ni DNS, ni Internet, ni le reste du réseau local.

Où cette règle doit s'écrire

Chemin d'un paquet d'attaque, et où le pare-feu le voit



Avec un honeypot conteneurisé, le trafic traverse la machine : il passe par forward.

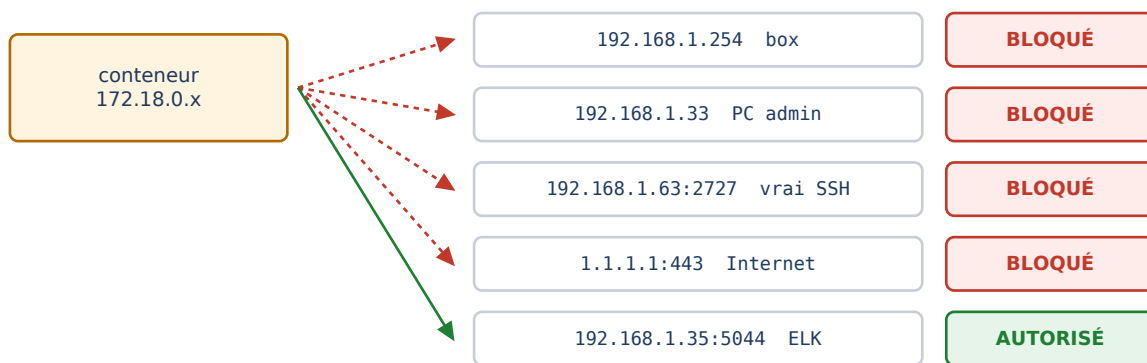
C'est le point technique que le TP 2 avait anticipé et que le TP 3 a vérifié : un conteneur ne passe **jamais** par la chaîne input ni par output. Une politique sortante écrite dans output ne filtre rien de ce que fait le honeypot.

Si le honeypot tourne...	Son trafic passe par	Où écrire la règle
directement sur l'hôte	input / output	chaîne output
dans un conteneur	forward	chaîne forward (et/ou DOCKER-USER)
dans une VM (pont réseau)	forward de l'hyperviseur	sur l'hyperviseur, ou un pare-feu en amont

La vérification, qui n'est pas facultative

Écrire la règle ne suffit pas : il faut **prouver** qu'elle fonctionne, depuis l'intérieur.

Tests d'isolation réalisés depuis l'intérieur du réseau du conteneur



Chaque refus est tracé : `dmesg | grep FWD-CONTAINER-DROP`

Le test mené sur la machine : cinq destinations, quatre refus, un seul passage.

```
# depuis un conteneur placé sur le même réseau que le honeypot
docker run --rm --network <reseau_du_honeypot> alpine sh -c '
  nc -z 192.168.1.254 80    # la box
  nc -z 192.168.1.33 22    # un autre poste
  nc -z 1.1.1.1 443       # Internet
'
# puis, côté hôte :
dmesg | grep FWD-CONTAINER-DROP
```

Un pare-feu qu'on n'a pas testé depuis l'intérieur de la zone qu'il est censé confiner n'est pas un pare-feu : c'est une intention. La règle qui ne matche pas ne prévient pas, et on ne s'en aperçoit qu'au moment où elle aurait dû servir.

4. La crédibilité du leurre

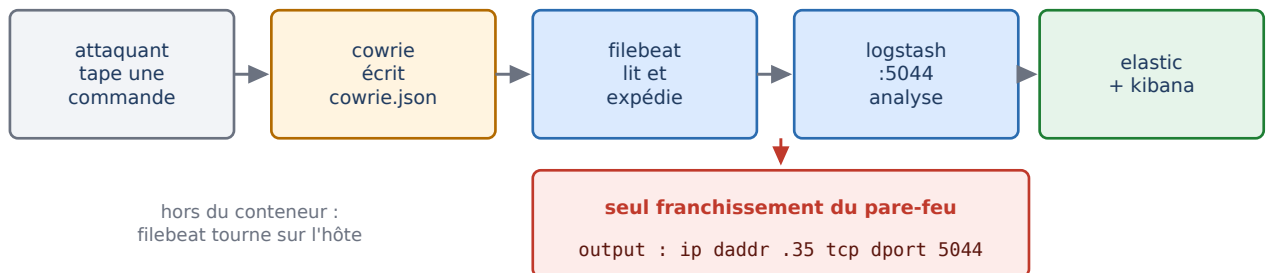
Un honeypot trop visible ne capture rien d'intéressant : l'attaquant part. Quelques détails font la différence, et ils sont tous gratuits.

Élément	À éviter	Préférer
Nom d'hôte	honeypot, cowrie, test	srv-prod-01, nas01 — un nom d'infrastructure ordinaire
Bannière	une version exotique ou incohérente	une version courante, légèrement ancienne — donc attirante
Identifiants	tout accepter — l'attaquant comprend immédiatement	une liste de mots de passe faibles réalistes ; le reste est refusé
Système de fichiers	l'arborescence par défaut, connue de tous	quelques fichiers personnalisés, un historique de commandes plausible
Temps de réponse	des réponses instantanées à des commandes coûteuses	cohérence avec le matériel annoncé

L'incohérence la plus fréquente, et elle est présente sur notre machine : le honeypot annonce OpenSSH_8.9p1 sur le port 22, tandis que le vrai serveur annonce OpenSSH_9.6p1 sur le port 2727. Un scan des deux ports révèle deux systèmes différents sur une même adresse. Aligner les bannières règle le problème — au prix d'un leurre moins attirant, puisqu'une version récente intéresse moins les robots. Les deux choix se défendent ; il faut le faire en connaissance de cause.

5. De la capture au renseignement

Le trajet d'un événement, de l'attaquant au tableau de bord



Un journal qu'on ne lit pas n'a aucune valeur. La chaîne de collecte fait partie du dispositif, pas de la décoration.

Pourquoi du JSON, et pourquoi le déclarer comme tel

Cowrie écrit un événement JSON par ligne. Si Filebeat expédie ces lignes comme du texte brut, Elasticsearch stocke une chaîne de caractères : on peut la chercher, pas l'analyser. Avec le greffon ndjson, chaque champ devient interrogeable :

```
parsers:  
- ndjson: { target: "", overwrite_keys: true }
```

La différence est celle-ci :

Sans parseur	Avec parseur
Un champ message contenant tout le JSON	src_ip, username, password, input... chacun interrogeable
« cherche les lignes contenant 192.168.1.5 »	« les 10 mots de passe les plus tentés, par heure, groupés par pays »

Les quatre questions auxquelles ton tableau de bord doit répondre

1. **Qui** : adresses sources, volume, évolution dans le temps.
2. **Quoi** : identifiants tentés, taux de réussite.
3. **Comment** : commandes saisies après une connexion réussie, dans l'ordre.
4. **Vers où** : URL de téléchargement tentées. C'est la donnée la plus précieuse, celle qu'on peut partager avec la communauté.

Un dispositif sans supervision de lui-même est incomplet. Si le honeypot tombe, le tableau de bord devient simplement vide — et un tableau vide ressemble à s'y méprendre à « rien ne se passe ». Prévois une alerte sur l'absence d'événements.

6. Avant d'exposer sur Internet

Le calendrier réel

Délai après ouverture du port 22	Ce qui arrive
quelques minutes	premiers scans automatisés — les plages d'adresses sont balayées en permanence
première heure	premières tentatives d'authentification par dictionnaire
premier jour	de l'ordre du millier de tentatives ; premières connexions réussies si tu acceptes des mots de passe faibles
première semaine	charges téléchargées, tentatives de persistance, recrutement pour botnet

La liste de contrôle

Point	Pourquoi
Sortant confiné et testé depuis l'intérieur	C'est le seul point réellement critique
Aucun accès du honeypot au reste du réseau	Le rebond est le scénario le plus probable
Le vrai SSH ailleurs, filtré par adresse, avec fail2ban	Ne pas perdre l'administration de la machine
fail2ban ne surveille pas les ports du honeypot	Bannir les attaquants reviendrait à saborder le capteur
Rotation des journaux et espace disque	Un port 22 exposé remplit un disque plus vite qu'on ne croit
Durée de conservation décidée	Les adresses IP sont des données à caractère personnel (RGPD)
Conditions de ton opérateur vérifiées	L'hébergement d'un service exposé peut être encadré
Supervision du honeypot lui-même	Savoir qu'il est tombé, et pas seulement que les journaux sont vides

Les aspects juridiques et éthiques

- **Tu es responsable du trafic sortant de ton adresse IP.** C'est la raison technique et juridique du confinement strict.
- **Collecter n'est pas attaquer.** Observer ce qui vient chez toi est légitime. Sonder en retour l'infrastructure de l'attaquant ne l'est pas.
- **Les adresses IP sont des données personnelles** au sens du RGPD. Un usage personnel d'apprentissage est couvert, mais la durée de conservation doit être limitée et décidée à l'avance.
- **Les échantillons récupérés sont du code malveillant réel.** Ils ne s'ouvrent pas sur une machine ordinaire. Dans notre configuration la question ne se pose pas : la sortie du conteneur étant fermée, aucun fichier n'est téléchargé — seule l'URL est enregistrée.

7. Pour aller plus loin

Piste	Intérêt
Tarpit SSH (endlesssh)	Répond une bannière infiniment lente : immobilise les robots
Autres protocoles	Dionaea (SMB, FTP), Conpot (systèmes industriels), ElasticHoney
Corrélation avec les listes publiques	Recouper les adresses sources avec des bases de réputation
Partage communautaire	Certains projets agrègent les données de honeypots ; à ne faire qu'en connaissance de cause
Honeypot à forte interaction	VM sacrificielle sur réseau totalement isolé — l'étape suivante, nettement plus exigeante

La leçon transversale de ce projet. La partie difficile n'a jamais été d'installer Cowrie — c'est une commande. Elle a été de garantir ce que le honeypot **ne peut pas** faire : les cinq bugs du TP 1, le piège du flush ruleset, le décalage des ports par le DNAT, l'exposition IPv6 involontaire. Dans un dispositif de sécurité, ce sont toujours les interdictions qui sont difficiles à écrire, et il faut les prouver plutôt que les supposer.